Source: docs\COUPONS.md | Export: 11 November 2026

Coupons administration

Canonical route: `/admin/coupons`. Desktop and mobile navigation place Coupons immediately after Marketing / Sale Campaigns and before Banners. Marketing retains `/admin/marketing` as its own unavailable campaign module; it no longer renders coupon management. Campaigns and banners are not implemented or moved by this change. Existing shell, logo, locally hosted Lora, 192px sidebar, approximately 58px header and compact density remain in use.

Management and scope

Components and pure helpers: `src/admin/coupons/`. The retired coupon form in `src/orders/AdminBusiness.tsx` is replaced by the dedicated list and editor. The new UI uses the existing `coupons` documents and trusted `/api/catalog` actions; no parallel coupon store is introduced.

- `coupons:list`: document-ID ordered, 25-record API pages. The UI completes a bounded read of at most 401 pages / 10,000 coupons before search, filtering, counts or five-coupon UI pagination. Any failed or inconsistent traversal prevents partial results from being displayed as complete.
- `coupons:get`: one authorized, normalized record for opening/reloading the editor.
- `coupons:save`: existing revision-checked transaction, with code normalized to uppercase and atomic create uniqueness via the expected revision. Existing codes are read-only in the editor. No deletion, duplication, export or bulk action is available.

List statuses use this precedence: Disabled, Expired (`now >= end`), Usage Limit Reached, Scheduled (`now < start`), Active. An enabled expired/exhausted coupon remains ineligible. Usage progress is shown only for finite total limits. Missing creation timestamps sort after known timestamps, then by code; no timestamp is invented. Dates and preview status update without rewriting coupon records.

Amounts, dates and existing coupons

Money is persisted as integer paise. Rupee entry accepts at most two decimal places. Percentage coupons retain whole percentages, integer floor rounding, and an optional cap applied once per order. Fixed discounts are clamped to the eligible product subtotal, never making totals negative. Minimum qualification uses product subtotal. Delivery remains separate, including the existing pre-coupon shipping qualification policy.

Date entry and display explicitly use Asia/Kolkata. `datetime-local` values are converted using the +05:30 offset, independently of browser/host timezone. Existing subsecond instants survive an unchanged edit. Start immediately and No end date persist null instants.

Missing optional legacy restrictions normalize as no restriction: start/end, percentage cap, total limit and per-customer limit become null; absent optional minimum becomes zero. Required code/type/value/enabled and invalid recorded usage never silently default. Legacy per-customer limits remain enforced and are preserved on edits, with a read-only explanation; the optional order-sequence restriction described below is supported; loyalty controls are unchanged. Corrupt settings fail explicitly instead of presenting partial coupons or resetting counters.

The preview shares `couponDiscount` with trusted checkout. It calculates eligibility, discount/cap and amount after discount for the entered subtotal. It is labelled **Preview - delivery charges not included.** It assumes zero prior uses for a particular customer and is advisory; actual customer limits and current usage are verified server-side. Preview and checkout review do not consume redemptions.

Concurrency, authorization and checkout

The management entry point requires both the `admin: true` claim and enabled `adminAccess/{uid}` record. Writes recheck enabled access transactionally. Ordinary customers cannot read/list/write admin coupon data; their existing checkout application flow is unchanged. Management responses project settings, revision, usage and timestamps without returning operation hashes/private fields.

Settings writes read the latest recorded usage inside the same transaction and merge only validated settings/metadata, preserving unknown unrelated fields. A concurrent checkout causes a retry against current usage instead of losing a redemption. No client-provided usage is accepted. Settings changes increment revision; checkout usage increments do not.

Exact failed-response settings retries keep the same operation ID/payload. A previously accepted identical last operation returns its recorded revision without resetting uses. A changed payload or intervening settings revision conflicts rather than reapplying stale settings. Editor conflicts preserve fields and require explicit Load latest settings and keep edits before saving. Duplicate creates retain fields and require a different code. Disabling requires confirmation. Dirty close/navigation prompts; window/tab focus changes do not reset drafts. Reads abort and private state unmounts on navigation/sign-out.

Final COD acceptance independently rechecks coupon existence/identity, enabled state, dates, minimum, current settings, usage limits and trusted product prices. The existing transaction coordinates redemption counts, stock deductions, order snapshots and operation receipts. Concurrent last-use attempts cannot both redeem. Accepted-order retries do not count twice. Changed settings return renewed review or explicit ineligibility rather than silently accepting a higher total. Cancellations retain usage under the existing policy. Accepted historical totals and coupon snapshots are not rewritten.

Verification and release

- `npm run lint`, `npm test`, `npm run build`.
- `src/admin/coupons/model.test.ts`: status boundaries, IST/money conversion, legacy omissions, shared preview arithmetic, complete-list traversal.
- `scripts/coupons-emulator.test.ts`: real coupon services and checkout, dual authorization, uniqueness/retries, concurrent redemptions/settings, preserved uses/private fields/history and API pagination.
- Existing business and order emulator regressions.
- `scripts/coupons-browser.test.py`: dedicated routes/navigation, list and editor, preview, IST persistence, stale/duplicate/uncertain operations, counts and responsive layouts. Fixture helpers require both exact loopback emulator endpoints.
- Existing business browser test updated for the new editor. New tests are registered in the existing admin emulator harness.

Accept locally with emulators at `/admin/coupons`: create a percentage and fixed coupon, preview qualifying/nonqualifying subtotals, schedule a coupon in IST, edit without changing usage, confirm/cancel disabling and inspect mobile filters. Use emulator customers/orders for redemption checks.

No new indexes, security rules, dependencies, migrations or production preparation are required. List pagination uses only document-name ordering. Owner application deployment and hosted acceptance remain release steps; local/emulator checks do not establish production readiness. Nothing was committed, pushed, deployed, published or changed in production.

Customer order sequence (November 2026)

In Create/Edit Coupon, choose **Customer eligibility > Specific order number** and enter a positive whole number (up to 1,000,000,000). Leave **All customers** selected to preserve unrestricted behaviour. The optional persisted `eligibleOrderNumber` defaults to unrestricted when missing/null. It appears in Admin list/details and round trips through revision-checked saves.

Example: THIRD with 3 requires exactly two `Delivered` orders owned by the authenticated token UID. COD collection is irrelevant; all other statuses are excluded. Review and final acceptance query `orders` using equality on `uid` and `status == Delivered`, project document names only, and limit to the entered number: reaching that number already proves ineligibility. Existing equality index merging supports this query; the exact shape was verified read-only against Production `(default)`. No new index/rule publication is needed.

Final acceptance reads both trusted Delivered records and the existing `coupons/{code}/customers/{uid}` usage record in the order transaction. Sequence coupons require zero prior accepted uses regardless of a looser configured per-customer limit. All existing dates, minimum, caps and total/per-customer limits still apply. Usage, stock, order and idempotency receipt commit atomically. Applying/reviewing, failed attempts and a changed review consume nothing. There is no separate coupon reservation. Exact accepted-operation retries recover the original order without another use. Accepted cancellations/rejections retain usage under the existing lifecycle, including sequence coupons; they do not release a use. Adding a restriction to an already-used coupon does not erase customer usage.

Regression coverage extends the existing coupon unit, real-service emulator and responsive Admin browser suites. No production customers, coupons or orders are created for testing.

Coupon and loyalty audit - November 2026 (local changes)

Customer discovery now requests all active, publicly listed coupons rather than eligible-only checkout results. `discoverable: true` is the existing publication switch; this model has no separate draft/published field. Private (`false` or missing), disabled, scheduled and expired codes stay hidden. Global/customer usage exhaustion and unmatched Delivered-order sequences remain visible as **Not Eligible**, with disabled Apply and a safe reason. Manual private-code entry remains supported and independently validated.

Optional `title` (100 characters) and `description` (2,000 characters, plain text with paragraph breaks) round trip through the existing Admin editor, schema, revision-checked saves and public projection. Legacy missing text stays valid; no sample descriptions or production backfill are added. Only these explicitly public text fields and the existing offer terms are projected, never usage records, customer identifiers, internal notes or operation evidence.

Product offers retain the compact, initially closed dropdown and display the selected coupon's full terms and description. Checkout retains its initially closed Available Coupons panel, now with both Eligible and Not Eligible cards and Copy for every displayed code. Applied invalid codes remain visible; the review shows the backend reason and blocks progression until the customer removes/replaces the code. Current items/session changes, focus/reconnect and the existing 15-second visible-page refresh recheck eligibility.

Shared server validation remains authoritative for dates, limits, subtotal, percentage/fixed arithmetic, caps and exact Delivered-order sequence. Review and discovery do not consume coupons; COD acceptance and Razorpay reservation revalidate inside the stock/loyalty transaction. Exact retries preserve the accepted operation. COD accepted cancellations retain coupon usage. Uncaptured online reservation expiry releases usage once. No rule or index changes.

Existing policies do not implement customer allowlists, product/category coupon targeting or an independent draft publication status. Per-customer usage and Delivered-order sequences are supported. Do not interpret unknown stored fields or client-supplied eligibility/discount values as a new targeting policy.

See COUPON_LOYALTY_AUDIT.md for the verification report and current limitations. These local changes require owner-approved release.